

Big-Step 증명나무 합성을 통한 정적 분석기 공격

정원준 지도교수 이광근 서울대학교 프로그래밍 연구실 ROPAS

“프로그램을 먼저 합성하면, 그 프로그램에 실행 의미가 무엇인지 알 수 없다.”

Rice's Theorem

01 분석기 공격이란

라이스 정리 Rice's Theorem 때문에 분석기는 안전성 · 유한성 · 정밀도 셋 중 하나는 반드시 포기한다.

안전성 soundness · 실제 값을 빠뜨리지 않음	유한성 termination · 엔진이 멈춤	정밀도 completeness · 쓸데없이 넓힘
---	------------------------------------	--------------------------------------

셋 다 만족하는 분석기는 없다. 우리는 앞의 돌을 지키는 분석기를 공격한다.

그러면 남는 것은 정밀도 하나다. 그것이 어느 지점에서 깨지는지를 사람 없이 자동으로 찾아내는 일이 곧 공격이다.

요약 결과가 모든 값 $\text{top} = (-\text{inf}, +\text{inf})$ 으로 무너지는 지점이 가장 좋은 공격

```
x = 1;
while (-x) { x = 0; }
x = x * x;
```

구체 실행 $x = 0$ 분석 결과 $x \mapsto [-\text{inf}, \text{inf}] = \text{top}$

조건식 guard이 x 가 아니라 $-x$ 라 끝난 뒤 걸러내기가 약하고, $x * x$ 는 두 항 operand이 같은 변수라는 관계를 잃는다. 이전 자체 분석기 엔진에서 찾은 예다.

03 공격의 정의

프로그램 하나에 실행이 여러 갈래로 갈리는 성질 nondeterminism을 없애야, 증명나무 proof tree가 하나로 정해지고 정밀도를 따지는 일이 뜻을 가진다.

CIL- · C 스타일 시키는 대로 도는 imperative 언어

대입 · 조건 · 반복 · 함수 호출(재 자신 부르기 recursion 포함)은 갖추되, 계산 순서나 정해두지 않은 동작 undefined behavior 같은 갈래는 문법에서 아예 잘라냈다.

이런 갈래가 남아 있으면 한 프로그램에 실행이 여러 개 대응된다. 그러면 공격에 성공한 것처럼 보여도 사실은 그 갈래 탓일 수 있다.

갈래를 없애면, 프로그램 하나에 증명나무가 딱 하나로 정해진다.

3 지점 L, 변수 x. 분석 결과(L, x) 구 실제 실행 의미(L, x)

어떤 프로그램 지점 label과 변수에서, 요약된 결과가 실제 실행 의미보다 진짜로 더 크다 strictly greater

진짜로 더 크다 = 어림잡은 값 안에, 실제로는 절대 나올 수 없는 값이 섞여 있다.

05 증명나무의 크기

앞에서 뿌리로 bottom-up 빠짐없이 지어 올리려면, '크기'를 무엇으로 잡을지가 핵심이다.

[LVar] $x \Rightarrow s0+0$	[EConst] $1 \Rightarrow 1$	[LVar] $x \Rightarrow s0+0$
[ISet] $x = 1; \Rightarrow \{x \mapsto 1\}$	[ELval] $x \Rightarrow 1$	
[SInstr] $\text{instr}[1] \Rightarrow \text{Normal}$	[SReturnSome] $\text{return } x; \Rightarrow \text{Return}(1)$	
[BSeq] $\text{block}[2] \Rightarrow \text{Return}(1)$		
[FReturn] $\text{main}() \Rightarrow \text{Return}(1)$		
[PMainReturn] $\text{main}() \Rightarrow 1$		

앞에서 뿌리로 - $\text{proof_size}(t) = 1 + \text{sum of premise sizes}$

마디 수 node만 세면?

실행되지 않는 갈림길 branch에 아무리 큰 코드가 들어가도 증명나무는 커지지 않는다. 같은 크기의 나무에 대응하는 프로그램이 무한히 많아진다.

프로그램 크기로 재면?

반대로 프로그램 크기는 그대로인데 실행 의미가 무한히 길어질 수 있다. 반복문 하나로 증명나무만 끝없이 자란다.

어느 쪽도 혼자서는 안 된다. 문법에 '구멍' hole을 들이는 이유다.

02 분석기 개발 사이클

공격은 개발 사이클의 한 단계다. 찾아낸 공격이 다음 분석기의 설계로 되먹임된다.



사람들이 실제로 많이 쓰는 코드 패턴에서 공격이 나오면, 그 공격은 더 좋은 분석기를 디자인하는 데 곧바로 기여한다.

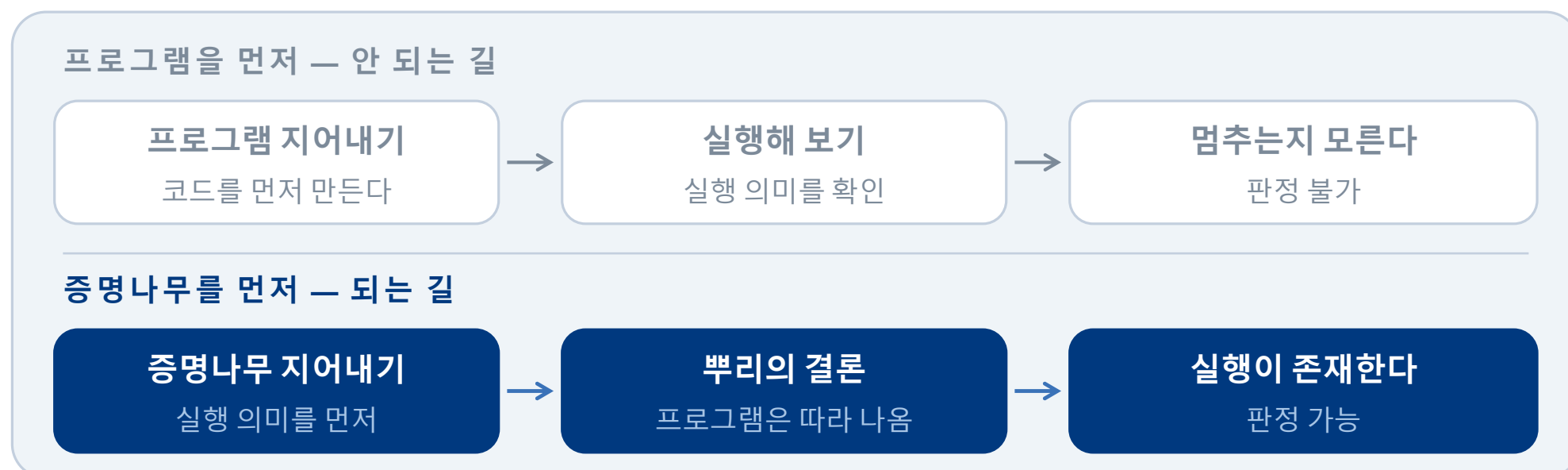
억지스러운 코드보다, 흔한 코드 패턴에서 나온 공격이 값지다

공격에서 디자인으로 돌아오는 고리가 닫혀야, 분석기가 실제로 좋아진다.

04 증명나무를 합성하는 이유

프로그램을 먼저 지어내면 synthesis, 그것이 공격에 성공했는지 판정할 수 없다.

공격 성공을 판정하려면 그 프로그램의 실제 실행 의미 concrete semantics를 알아야 한다. 그런데 프로그램이 멈추는지조차 미리 알 수 없다 (라이스 정리). 쉽게 말해, 무한히 돌지도 모른다.



유한한 증명나무가 있다는 것은, 곧 그 실행이 존재한다는 뜻이다.

06 구멍 뚫린 증명나무와 짝맞추기

실행되지 않은 자리를 구멍 hole으로 남기면, 크기를 그냥 마디 수로 정의할 수 있다.

```
if (x || ?H1) {
  return 0;
} else {
  ?H2
}
```

?H1 앞에서 이미 결판나 계산하지 않은 오른쪽 항

?H2 선택되지 않은 갈림길, 또는 return 뒤에 잘린 문장 꼬리

구멍이 놓일 수 있는 자리 — 이 셋뿐

- if 문의 선택되지 않은 갈림길 branch
- 차례로 이어진 문장 sequence의 맨 끝 — `return` · `break` · `continue` 뒤
- `&&` / `||` 에서 앞에서 결판나 short-circuit 계산하지 않은 오른쪽 항

올바른 증명나무에서 구멍은 절대 실행되지 않는다

짝맞추기 unification — 왜 필요한가

반복문과 재 자신을 부르는 함수에서는 같은 자리가 여러 번 실행된다. 그때 나온 조각들은 서로 실제로 같아야 한다. 타입 추론 type inference에서 쓰는 것과 같은 짝맞추기로 맞추고, 나무와 갈아끼우기 substitution를 따로 들고 다닌다.



SEOUL
NATIONAL
UNIVERSITY



한국정보과학회 프로그래밍언어연구회



Programming
Research Laboratory